Chapter 4: Booleans and Comparisons

An expression can now answer a question. Is the input large enough? Are two names equal? The result is a boolean: one of the two truth values, true or false. (1)

Vertex already computes numbers and strings. This version adds boolean literals, comparisons, explicit conversions, and a type query. The tokenizer recognizes their spellings. The parser establishes which operations belong together. The evaluator computes the result. Combining truth values with logical operators comes next in Logical Operators.

Breaking Changes

Earlier use	Change needed
Variables named `true` or `false`	Rename them; these are now literal keywords.
Variables named `boolean` or `type`	Rename them; these are now conversion and type-query keywords.

The conversion contracts also broaden: `number(42)` and `string("text")` now return their arguments instead of raising type errors. Existing valid conversions keep their results.

Longer names such as `true_value`, `boolean_value`, and `typewriter` remain identifiers. The keyword patterns require a word boundary.

Strings and Basic I/O remains the base. Floating-point literals, unary minus, comments, assignments, and conversions still work. Print still requires parentheses. Statements still require semicolon separators. `__input` supplies one input value and is cleared when consumed; `__output` captures the latest printed text.

A Program That Computes a Truth Value

```
__input = "21";
answer = number(input()) * 2;
large_enough = answer >= 40;
print(large_enough);
print("Captured: " + __output);
```

The output is:

```
true
Captured: true
```

The first assignment supplies input so the program does not wait for a keyboard line. The next computes `42`. Comparing it with `40` produces true. Assignment stores that boolean under `large_enough`, just as it would store a number or string. Print writes lowercase `true` and stores that same text in `__output`.

The code is in `chapter-04-booleans-and-comparisons/code`. From that directory:

```
./vertex example.v < /dev/null
bash run-tests.sh
```

The supplied example exercises all six comparisons, conversions, and `type()`. It remains non-interactive. The test script checks its complete output.

Boolean Literals and Values

`true` and `false` are literal values, not names that need environment entries. Their abstract syntax tree (AST) nodes carry a boolean value:

```
{"tag": "boolean", "value": True}
{"tag": "boolean", "value": False}
```

The capitalized values here belong to Python, the implementation language. Vertex source uses lowercase keywords. The parser creates the node directly:

```
if token["tag"] in ("true", "false"):
    return {"tag": "boolean", "value": token["tag"] == "true"}, tokens[1:]
```

The evaluator returns the node's value. An identifier such as `large_enough` instead looks up its value in the environment. Both routes can supply a boolean operand.

Booleans are distinct from numbers in this Vertex version. Python permits some integer operations on its boolean values; we deliberately reject those uses. `true + 1`, `-false`, and `"ha" * true` raise type errors. The existing `string` conversion now accepts booleans explicitly: `string(true)` returns `"true"`. Print can display a boolean without that conversion.

Converting Values Explicitly

Each conversion accepts one expression. If its value already has the requested type, the conversion returns it unchanged.

Input value	`number(value)`	`string(value)`	`boolean(value)`
`42`	`42`	`"42"`	`true`
`0.0`	`0.0`	`"0.0"`	`false`
`true`	`1`	`"true"`	`true`
`false`	`0`	`"false"`	`false`
`"42"`	`42`	`"42"`	Value error
`"false"`	Value error	`"false"`	`false`

Numeric-to-boolean conversion treats zero as false and every nonzero value as true, including negative numbers. String-to-boolean conversion accepts only `true` or `false`, ignoring surrounding whitespace and case. Thus `boolean(" FALSE ")` is false. Empty text, `"yes"`, `"0"`, and `"1"` raise value errors rather than silently choosing a meaning. To interpret numeric text as a boolean, write `boolean(number("1"))`.

String-to-number conversion retains the signed decimal syntax from Strings and Basic I/O. It ignores surrounding whitespace but rejects `"true"`, exponent notation, and arithmetic expressions. Number-to-string conversion uses the host's numeric formatting; very small or large floats may produce exponent notation, so a number-to-string-to-number round trip is not guaranteed by the current decimal-only reader.

The conversions do not introduce implicit coercion. `true + 2` still fails, but `number(true) + 2` produces `3`. `boolean(1)` explicitly produces a boolean; it does not change the rule that `true == 1` is false.

Asking for a Value's Type

`type(expression)` returns one of three strings: `"number"`, `"string"`, or `"boolean"`. Integers and floats both belong to Vertex's number family.

```
print(type(42));
print(type(.5));
print(type("42"));
print(type(1 < 2));
print("Accepted: " + string(boolean(-2)));
```

The output is `number`, `number`, `string`, `boolean`, and `Accepted: true`, each on its own line. The result of `type` is itself a string, so `type(type(false))` returns `"string"`.

The argument is evaluated exactly once before its type is inspected. This is a query about a value, not source syntax. `type(input())` reads input and returns `"string"`; supplied input is consumed and cleared normally. `type(missing)` still raises an unknown-identifier error.

The parser treats these as dedicated expression forms, not general function calls:

```
number_expression ::= "number" "(" expression ")"
string_expression ::= "string" "(" expression ")"
boolean_expression ::= "boolean" "(" expression ")"
type_expression ::= "type" "(" expression ")"
```

All four become factor alternatives and therefore work inside larger expressions. Their AST nodes keep one `expression` child; the tags are `number_conversion`, `string_conversion`, `boolean_conversion`, and `type_query`. The evaluator handles Python booleans separately from integers so that `type(true)` reports `"boolean"`, not `"number"`.

Comparing Values

Relational operators compare operands and produce truth values. Languages differ in which types they permit and how they treat mixed types. (2) Vertex uses these rules:

Operator	Meaning	Example	Result
`==`	Equal	`42 == 42.0`	`true`
`!=`	Unequal	`"dog" != "cat"`	`true`
`<`	Less than	`-1.5 < .5`	`true`
`<=`	Less than or equal	`2 <= 2`	`true`
`>`	Greater than	`2 > 3`	`false`
`>=`	Greater than or equal	`3 >= 3`	`true`

Equality accepts numbers, strings, and booleans. Integers and floats share numeric equality: `1 == 1.0` is true. Other different types compare unequal. Thus `true == 1` and `"1" == 1` are both false. No operand is silently converted.

The four ordering operators accept either two numbers or two strings. Strings use lexicographic order: compare characters from left to right; the first unequal pair decides the result. If one string is a prefix of the other, the shorter string comes first. Equal strings satisfy <= and >=, but not < or >.

Expression	Result
`"ant" < "bee"`	`true`
`"ant" < "anteater"`	`true`
`"" < "a"`	`true`
`"ant" <= "ant"`	`true`
`"Z" < "a"`	`true`
`"10" < "2"`	`true`

Character order follows Unicode code points, as in Python's string comparison. It is case-sensitive, not a locale-specific dictionary order. Numeric-looking strings are still text; use `number` to compare their numeric values instead. `false < true` and `"1" < 2` raise type errors. No implicit conversion is added.

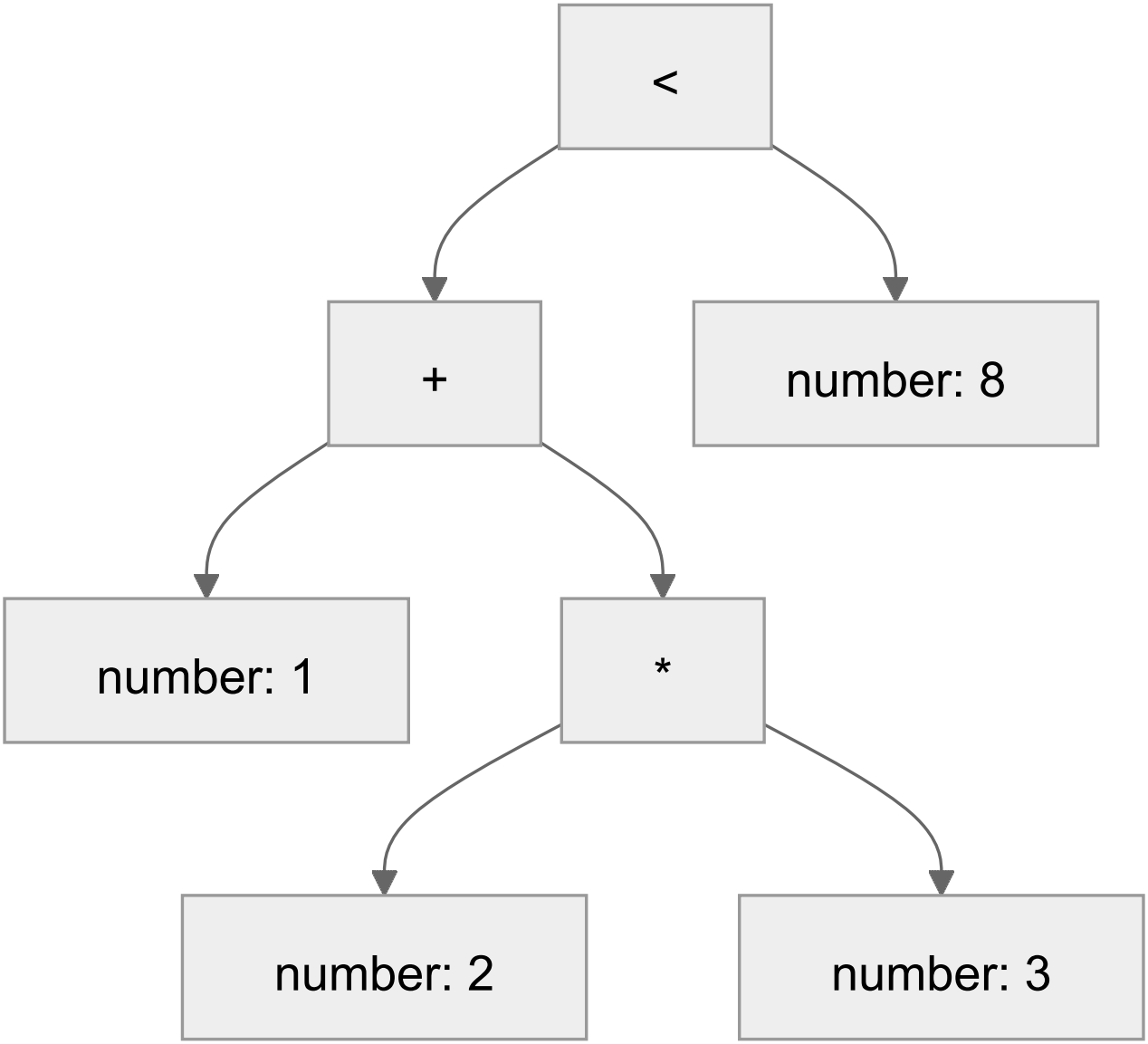
The evaluator first evaluates the left operand, then the right. Equality checks whether their types are compatible before comparing their values. Ordering checks for two exact numeric types (`int` or `float`) or two `str` values. Using exact types prevents Python's `bool` subclass of `int` from passing the numeric check. The existing operator cases then handle either pair; strings need no new tokens, parser rules, or tree nodes.

Comparison Precedence and the Parser

Precedence determines grouping when parentheses do not specify it. (3) Comparisons bind less tightly than arithmetic. In `1 + 2 * 3 < 8`, the left operand is the arithmetic expression `1 + 2 * 3`, not just `3`.

Weakest to strongest	Operators
Comparison	==, !=, <, <=, >, >=
Addition and subtraction	+, −
Multiplication and division	*, /
Negation	Unary −

The abstract syntax tree records that grouping:



Extended Backus-Naur Form (EBNF) makes the new level explicit:

```
expression ::= comparison
comparison ::= arithmetic_expression [ compare_op arithmetic_expression ]
compare_op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
arithmetic_expression ::= term { ("+" | "-") term }
term ::= unary { ("*" | "/") unary }
unary ::= "-" unary | factor
```

The previous arithmetic rule is now `parse_arithmetic_expression`. `parse_expression` calls `parse_comparison`, which asks the arithmetic parser for each operand. A parenthesized factor reenters `parse_expression`, allowing a comparison inside parentheses.

```
def parse_comparison(tokens):
    """comparison ::= arithmetic_expression [ compare_op arithmetic_expression ]"""
    # Arithmetic binds more tightly than comparisons: 1 + 2 < 4 compares 3 to 4.
    left, tokens = parse_arithmetic_expression(tokens)
    # One optional operator, not a loop: unparenthesized comparison chains are
    # deliberately outside this grammar. The unused token will cause an error.
    if tokens[0]["tag"] in ("==", "!=", "<", "<=", ">", ">="):
        operator = tokens[0]["tag"]
        right, tokens = parse_arithmetic_expression(tokens[1:])
        return {"tag": operator, "left": left, "right": right}, tokens
    return left, tokens
```

One optional suffix means one comparison, not a chain. `1 < 2 < 3` is rejected. Explicit nesting such as `(1 < 2) == true` is valid. Parentheses settle grouping, not types: `(1 < 2) < 3` still fails because booleans cannot be ordering operands.

The tokenizer must match == before = and <= before <. Otherwise it would split one operator into two tokens before the parser ever saw it.

Tests and Error Boundaries

The tests retain valid behavior from Strings and Basic I/O and update the conversion expectations where the contracts broaden. New cases check all six comparisons, exact boolean result types, precedence, conversions, and `type()`. Input tests verify that comparison operands run left to right and conversion arguments run exactly once.

Program fragment	Result or error	Stage
`x = 1 + 2 * 3 == 7`	Stores `true`	Evaluation
`x = 1 < 2 < 3`	Extra comparison operator	Parsing
`x = true + 1`	Boolean arithmetic operand	Evaluation
`x = "a" < "b"`	Stores `true`	Evaluation
`x = "1" < 2`	Mixed string and number operands	Evaluation
`x = boolean("yes")`	Invalid boolean text	Evaluation
`x = type(input())`	Reads input; stores `"string"`	Evaluation

The runner still reports errors as Python tracebacks. There are no logical operators, conditionals, or assertion statements in this version. Tests use Python assertions and the existing `__input`/`__output` behavior.

Considerations

1. What would change if booleans automatically acted as numbers in arithmetic?
2. How does case-sensitive character ordering differ from sorting words in a dictionary?
3. Should `boolean("0")` be accepted? How would that choice affect the conversion rules?
4. What extra semantics would support a comparison chain such as `low < value < high`?

Practice

Answers to the practice questions follow this section.

1. Supply numeric input through `__input`, compare it with a threshold, and check the printed text in `__output`.
2. Draw the tree for `1 + 2 * 3 == 7`. Change the parentheses and predict the result.
3. Compare `true == 1` with `number(true) == 1`. Locate the evaluator rules responsible for the difference.
4. Try `boolean("false")`, `boolean("0")`, and `boolean(number("0"))`. Explain which conversions succeed and why.
5. Add a test confirming that `type(input())` consumes supplied input once.

Answers to the Practice Questions

1. A threshold test can run with supplied input and inspect the output:

```
from evaluator import evaluate_source

_, environment = evaluate_source(
    'print(number(input()) >= 60)', {"__input": "72"})
assert environment["__output"] == "true"
```

2. The root is `==`; its left child is `+`, whose right child is `*`. Multiplication produces `6`, addition produces `7`, and equality produces `true`. `(1+2)*3 == 7` instead has a multiplication below the equality and produces `false`, because its left value is `9`.

3. `true == 1` is `false`; `number(true) == 1` is `true`. Equality excludes booleans from numeric equality. The explicit number conversion produces integer `1`, so the second comparison has two numeric operands.

4. `boolean("false")` returns `false`. `boolean("0")` raises `ValueError`: a boolean string must spell true or false, ignoring case and surrounding whitespace. `boolean(number("0"))` returns `false`, because the explicit number conversion first produces zero.

5. A single supplied read leaves the input empty and reports the string value's type:

```
from unittest.mock import patch
from evaluator import evaluate_source

with patch("builtins.input") as read:
    _, environment = evaluate_source(
        'kind=type(input())', {"__input": "Ada"})
read.assert_not_called()
assert environment["kind"] == "string"
assert environment["__input"] == ""
```

References

1. Wikipedia contributors. "Boolean data type." Wikipedia. https://en.wikipedia.org/wiki/Boolean_data_type
2. Wikipedia contributors. "Relational operator." Wikipedia. https://en.wikipedia.org/wiki/Relational_operator
3. Wikipedia contributors. "Order of operations." Wikipedia. https://en.wikipedia.org/wiki/Order_of_operations